Практичне заняття № 2

Тема: Робота з статичними даними і методами

Мета заняття: закріпити навички створення класів, об’єктів, полів і методів у Java; навчитися використовувати статичні поля та методи, константи static final; ознайомитися з перерахуваннями enum, їх використанням у класах, конструкціях switch та методом values(); повторити конструктори, масиви об’єктів, рядки, умови й цикли.

Короткі теоретичні положення

Клас і об’єкт. Клас описує структуру та поведінку майбутніх об’єктів. Поля зберігають дані, методи виконують дії. Об’єкт створюється за допомогою оператора new.

Метод. Метод – це іменований блок коду, який виконує певну дію. Метод може отримувати параметри. Якщо метод повертає результат, указують тип результату та використовують return. Якщо результат не повертається, використовують void.

Статичне поле. Поле з модифікатором static належить класу, а не окремому об’єкту. Для всіх об’єктів класу існує одна спільна копія такого поля. Типовий приклад – лічильник створених об’єктів.

Статичний метод. Метод static можна викликати через ім’я класу без створення об’єкта. Статичний метод безпосередньо працює зі статичними членами класу.

Константа static final. Поєднання static final використовують для значень, спільних для всіх об’єктів і незмінних під час виконання програми. Назви таких констант зазвичай записують великими літерами.

Метод main. Метод public static void main(String[] args) є точкою входу в програму. static дає змогу JVM викликати main без попереднього створення об’єкта класу.

Перерахування enum

enum (перерахування) – це спеціальний тип у Java для набору наперед визначених констант. Він зручний, коли значення можуть належати лише до обмеженого переліку: пора року, жанр, категорія товару, пріоритет заявки, статус замовлення тощо.

```
enum Season {
    WINTER,
    SPRING,
    SUMMER,
    AUTUMN
}
```

Значення задають через ім’я типу, наприклад Season.SUMMER. enum можна використовувати як тип поля класу, параметр методу та у switch. Метод values() повертає всі значення перерахування.

Приклад enum у класі

```
enum Genre {
    PROGRAMMING,
    SCIENCE,
    FICTION
}

class Book {
    String title;
    Genre genre;

    Book(String title, Genre genre) {
        this.title = title;
        this.genre = genre;
    }
}
```

Приклад switch з enum

```
static String genreName(Genre genre) {
    switch (genre) {
        case PROGRAMMING:
            return "Програмування";
        case SCIENCE:
            return "Наукова";
        case FICTION:
            return "Художня";
        default:
            return "Невідомий жанр";
    }
}
```

Приклади

1. Звичайне поле і статичне поле

```
class Student {
    String name;          // власне поле кожного об'єкта
    static int count = 0; // спільне поле класу

    Student(String name) {
        this.name = name;
        count++;
    }
}
```

2. Звичайний метод і статичний метод

```
class Calculator {
    int square(int x) {
```

```
        return x * x;
    }

    static int sum(int a, int b) {
        return a + b;
    }
}

// Виклик:
Calculator c = new Calculator();
System.out.println(c.square(5));
System.out.println(Calculator.sum(5, 7));
```

3. Константа класу

```
class Shop {
    static final double DISCOUNT = 0.10;
}
```

Хід виконання роботи та реєстрація результатів

Практична робота побудована як невеликий проєкт. За своїм варіантом необхідно створити клас предметної області (наприклад, Student, Book, Product, Movie), кілька об’єктів цього класу та виконати їх опрацювання.

- створити власне перерахування enum відповідно до предметної області;
- створити окремий клас предметної області;
- додати не менше двох звичайних полів об’єкта;
- додати щонайменше одне поле static, спільне для всіх об’єктів;
- додати одну константу static final;
- реалізувати конструктор з параметрами;
- реалізувати щонайменше один звичайний метод об’єкта;
- реалізувати щонайменше один static-метод із параметрами та результатом через return;
- створити не менше трьох об’єктів;
- помістити об’єкти в масив і переглянути його за допомогою циклу;
- використати хоча б одну умову if та одну операцію з рядком;
- вивести результати роботи програми з пояснювальними підписами.

Приклад виконання завдання

Завдання. Створити програму «Мінімагазин книг». Для кожної книги зберігати назву та ціну. Статичне поле повинно рахувати кількість створених книг. Константа задає знижку. Звичайний метод виводить інформацію про книгу, а статичний метод обчислює ціну зі знижкою. Книги зберегти в масиві та знайти книги, у назві яких є слово Java.

```
class Book {
    String title;
    double price;

    static int count = 0;
    static final double DISCOUNT = 0.10;

    Book(String title, double price) {
        this.title = title;
        this.price = price;
        count++;
    }

    void showInfo() {
        System.out.println(title + " — " + price + " грн");
    }

    static double priceWithDiscount(double price) {
        return price * (1 - DISCOUNT);
    }
}

public class Main {
    public static void main(String[] args) {
        Book[] books = {
            new Book("Java для початківців", 500),
            new Book("Алгоритми", 420),
            new Book("Java OOP", 650)
        };

        System.out.println("Кількість створених книг: " +
Book.count);

        for (Book book : books) {
            book.showInfo();

            if (book.title.contains("Java")) {
                System.out.println(
                    "  Ціна зі знижкою: "
                    + Book.priceWithDiscount(book.price) + " грн"
                );
            }
        }
```

```
    }
}
```

Розширення прикладу за допомогою enum

```
enum Genre {
    PROGRAMMING,
    SCIENCE,
    FICTION
}

// У класі Book:
Genre genre;

// У конструкторі:
Book(String title, double price, Genre genre) {
    this.title = title;
    this.price = price;
    this.genre = genre;
    count++;
}

// Створення об'єкта:
Book book = new Book(
    "Java для початківців",
    500,
    Genre.PROGRAMMING
);
```

Зверніть увагу:

- “title” і “price” мають окремі значення для кожної книги;
- “count” один для всього класу Book і збільшується в конструкторі;
- “DISCOUNT” – спільна незмінна константа;
- “showInfo()” викликається для конкретного об’єкта;
- “priceWithDiscount()” викликається через ім’я класу Book;
- масив містить не рядки, а об’єкти Book;
- “contains("Java")” повторює роботу з методами класу String.

Невеликий експеримент

Перед виконанням свого варіанта запустіть приклад, а потім послідовно виконайте три зміни. Після кожної зміни спрогнозуйте результат ДО запуску програми, а потім перевірте себе:

1. додайте четверту книгу та перевірте значення Book.count;
2. змініть DISCOUNT з 0.10 на 0.20 і порівняйте результат;

3. спробуйте звернутися до статичного поля як Book.count і як books[0].count. Обидва записи компілюються, але поясніть, чому для static правильніше використовувати ім’я класу.

Варіанти індивідуальних завдань

Варіант	Мініпроєкт	Завдання
B.1	**Студентський рейтинг**	Student: ім’я, бал. static count. Константа PASS_SCORE. Метод show(). static isPassed(ball). Створити масив студентів, вивести тих, хто склав, і порахувати їх. Додати enum StudyForm (BUDGET, CONTRACT) і використати його як поле класу.
B.2	**Бібліотека**	Book: назва, автор, ціна. static count. Константа DISCOUNT. Метод showInfo(). static finalPrice(price). Вивести книги заданого автора та ціну зі знижкою. Додати enum Genre (PROGRAMMING, SCIENCE, FICTION) і використати його як поле класу.
B.3	**Кав’ярня**	Drink: назва, ціна. static count. Константа SERVICE_FEE. Метод show(). static totalPrice(price). Вивести напої дорожчі за задану суму та їх кінцеву ціну. Додати enum Size (SMALL, MEDIUM, LARGE) і використати його як поле класу.
B.4	**Кінотеатр**	Movie: назва, тривалість. static count. Константа LONG_MOVIE_MINUTES. Метод showInfo(). static isLong(minutes). Вивести довгі фільми та їх кількість. Додати enum Genre (COMEDY, DRAMA, ACTION) і використати його як поле класу.
B.5	**Інтернет-магазин**	Product: назва, ціна. static count. Константа VAT. Метод show(). static priceWithVat(price). Знайти товари, назви яких починаються із заданої літери. Додати enum Category (ELECTRONICS, CLOTHES, BOOKS) і використати його як поле класу.
B.6	**Фітнес-клуб**	Member: ім’я, кількість відвідувань. static count. Константа ACTIVE_LIMIT. Метод show(). static isActive(visits). Вивести активних відвідувачів. Додати enum Membership (BASIC, STANDARD, PREMIUM) і використати його як поле класу.
B.7	**Музичний плейлист**	Song: назва, виконавець, тривалість. static count. Константа LONG_SONG. Метод show(). static isLong(duration). Вивести довгі композиції певного виконавця. Додати enum MusicGenre (POP, ROCK, CLASSICAL) і використати його як поле класу.
B.8	**Склад товарів**	Item: назва, кількість. static count. Константа MIN_STOCK. Метод show(). static needsRestock(quantity). Вивести товари, які потрібно поповнити. Додати enum Category (FOOD, TECHNIQUE, OTHER) і використати його як поле класу.
B.9	**Онлайн-курс**	Course: назва, кількість студентів. static count. Константа GROUP_LIMIT. Метод show(). static isFull(students). Вивести заповнені курси та курси зі словом Java. Додати enum Level (BEGINNER, INTERMEDIATE, ADVANCED) і використати його як поле класу.

B.10	**Таксі**	Ride: пункт призначення, відстань. static count. Константа PRICE_PER_KM. Метод show(). static calculatePrice(km). Вивести поїздки довші за задану відстань та їх вартість. Додати enum RideType (ECONOMY, COMFORT, BUSINESS) і використати його як поле класу.
B.11	**Готель**	Room: номер, ціна, тип. static count. Константа DISCOUNT. Метод show(). static finalPrice(price). Вивести кімнати заданого типу та ціну зі знижкою. Додати enum RoomType (STANDARD, DELUXE, SUITE) і використати його як поле класу.
B.12	**Зоопарк**	Animal: ім'я, вид, вік. static count. Константа ADULT_AGE. Метод show(). static isAdult(age). Вивести дорослих тварин заданого виду. Додати enum AnimalType (MAMMAL, BIRD, REPTILE) і використати його як поле класу.
B.13	**Смартфони**	Phone: модель, ціна. static count. Константа EXPENSIVE_PRICE. Метод show(). static isExpensive(price). Вивести дорогі моделі та моделі, назва яких містить задане слово. Додати enum PhoneType (BUDGET, MIDRANGE, FLAGSHIP) і використати його як поле класу.
B.14	**Ресторан**	Dish: назва, ціна. static count. Константа SERVICE_PERCENT. Метод show(). static total(price). Вивести страви дешевші за задану суму та кінцеву ціну. Додати enum DishType (STARTER, MAIN, DESSERT) і використати його як поле класу.
B.15	**Банк**	Account: власник, баланс. static count. Константа MIN_BALANCE. Метод show(). static hasEnough(balance). Вивести рахунки з достатнім балансом. Додати enum AccountType (DEBIT, CREDIT, SAVINGS) і використати його як поле класу.
B.16	**Комп'ютерні ігри**	Game: назва, жанр, рейтинг. static count. Константа HIGH_RATING. Метод show(). static isRecommended(rating). Вивести рекомендовані ігри заданого жанру. Додати enum GameGenre (ACTION, STRATEGY, PUZZLE) і використати його як поле класу.
B.17	**Аеропорт**	Flight: напрямок, ціна. static count. Константа AIRPORT_FEE. Метод show(). static totalPrice(price). Вивести рейси, напрямок яких починається із заданої літери. Додати enum FlightStatus (ON_TIME, DELAYED, CANCELLED) і використати його як поле класу.
B.18	**Зарплата працівників**	Employee: ім'я, зарплата. static count. Константа BONUS_LIMIT. Метод show(). static hasBonus(salary). Вивести працівників, які отримують бонус. Додати enum Position (JUNIOR, MIDDLE, SENIOR) і використати його як поле класу.
B.19	**Автосалон**	Car: модель, ціна, рік. static count. Константа NEW_CAR_YEAR. Метод show(). static isNew(year). Вивести нові автомобілі та моделі, назви яких містять заданий фрагмент. Додати enum CarType (SEDAN, SUV, ELECTRIC) і використати його як поле класу.

B.20	**Система заявок**	Ticket: тема, пріоритет. static count. Константа HIGH_PRIORITY. Метод show(). static isUrgent(priority). Вивести термінові заявки та заявки, тема яких містить слово Error. Додати enum Priority (LOW, MEDIUM, HIGH) і використати його як поле класу.

Додаткове завдання «Знайди помилку»

Поясніть, чому наведений код не можна використати в такому вигляді, і виправте його.

```
class Counter {
    int value = 10;

    static void printValue() {
        System.out.println(value);
    }
}
```

Додаткове завдання «Що виведе програма?»

```
class User {
    static int count = 0;

    User() {
        count++;
    }
}

public class Main {
    public static void main(String[] args) {
        User a = new User();
        User b = new User();
        User c = new User();

        System.out.println(User.count);
    }
}
```

Запишіть відповідь до запуску програми та поясніть її одним-двома реченнями.

До звіту необхідно включити

- тему та мету практичного заняття;
- умову свого індивідуального варіанта;
- лістинг класу предметної області та класу Main;
- скріншот результату виконання програми;
- коротке пояснення, які члени класу є звичайними, а які static;
- опис створеного enum та пояснення, де і навіщо він використовується;

- пояснення призначення static final у програмі;
- відповідь на завдання «Знайди помилку» або «Що виведе програма?»;
- висновки.

У висновках потрібно

- пояснити різницю між полем об’єкта і статичним полем класу;
- зазначити, які методи були реалізовані та як вони викликалися;
- описати, як використовувався конструктор;
- пояснити призначення enum та навести використане значення перерахування;
- пояснити роль static final;
- коротко проаналізувати результат роботи програми.

Контрольні питання

1. Що таке клас і що таке об’єкт?
2. Для чого в класі використовуються поля?
3. Що таке метод?
4. Чим параметр методу відрізняється від аргументу під час виклику?
5. Для чого використовується return?
6. Коли метод оголошують з типом void?
7. Для чого потрібен конструктор?
8. Що означає ключове слово this у конструкторі?
9. Що означає модифікатор static?
10. Кому належить статичне поле — класу чи об’єкту?
11. Скільки копій статичного поля існує для всіх об’єктів класу?
12. Як правильно звернутися до статичного поля?
13. Чим статичний метод відрізняється від звичайного методу об’єкта?
14. Чи потрібно створювати об’єкт для виклику static-методу?
15. Чому метод main оголошено static?
16. Чи може static-метод безпосередньо звертатися до нестатичного поля об’єкта?
17. Для чого використовується final?
18. Що означає поєднання static final?
19. Як створити масив об’єктів?
20. Як викликати звичайний метод для кожного об’єкта в масиві?

Додаткові контрольні питання з enum

1. Що таке enum у Java?
2. У яких випадках доцільно використовувати enum замість String або int?
3. Як оголосити власне перерахування?
4. Як звернутися до конкретного значення enum?

5. Чи можна використовувати enum як тип поля класу?
6. Як використати enum у конструкції switch?
7. Для чого використовується метод values()?
8. Що повертає метод name() для значення enum?
9. Чим enum відрізняється від набору звичайних рядкових констант?
10. Наведіть приклад enum для власної предметної області.